

Nodes

A node is designed to perform one logical task

Nodes can communicate with other nodes within the same process, in a different process or in a different machine.

Nodes use client libraries to communicate.

You can configure the nodes while it is running using parameters

Discovery

When a new worker (node) joins the factory floor, they don't need a central directory to find others.

1. **Announcement:** The new node announces its presence to others in the same **ROS Domain** (like being on the same company Wi-Fi, determined by the `ROS_DOMAIN_ID` variable).
2. **Handshake:** Other nodes respond with their information, and connections are made automatically.
3. **Compatibility Check:** Connections are only established if nodes have compatible **Quality of Service (QoS)** settings.
 1. Nodes periodically advertise their presence so that connections can be made with new-found entities, even after the initial discovery period.
 2. Nodes advertise to other nodes when they go offline.

Nodes will only establish connections with other nodes if they have compatible **Quality of Service** settings.

Take the [talker-listener demo](#) for example. Running the C++ talker node in one terminal will publish messages on a topic, and the Python listener node running in another terminal will subscribe to messages on the same topic.

Interfaces

Background

ROS applications typically communicate through interfaces of one of three types: [topics](#), [services](#), or [actions](#). ROS 2 uses a simplified description language, the interface definition language (IDL), to describe these interfaces. This description makes it easy for ROS tools to automatically generate source code for the interface type in several target languages.

In this document we will describe the supported types:

- **msg:** `.msg` files are simple text files that describe the fields of a ROS message. They are used to generate source code for messages in different languages.
- **srv:** `.srv` files describe a service. They are composed of two parts: a request and a response. The request and response are message declarations.
- **action:** `.action` files describe actions. They are composed of three parts: a goal, a result, and feedback. Each part is a message declaration itself.

Messages

Messages are a way for a ROS 2 node to send data on the network to other ROS nodes, with no response expected. For instance, if a ROS 2 node reads temperature data from a sensor, it can then publish that data on the ROS 2 network using a `Temperature` message. Other nodes on the ROS 2 network can subscribe to that data and receive the `Temperature` message.

Messages are described and defined in `.msg` files in the `msg/` directory of a ROS package. `.msg` files are composed of two parts: fields and constants.

Fields

Each field consists of a type and a name, separated by a space, i.e:

```
fieldtype1 fieldname1
fieldtype2 fieldname2
fieldtype3 fieldname3
```

For example:

```
int32 my_int
string my_string
```

Field types

Field types can be:

- a built-in-type
- names of Message descriptions defined on their own, such as “geometry_msgs/PoseStamped”

Built-in-types currently supported:

Type name	C++	Python	DDS type
bool	bool	builtins.bool	boolean
byte	uint8_t	builtins.bytes*	octet
char	char	builtins.int*	char

float32	float	builtins.float*	float
float64	double	builtins.float*	double
int8	int8_t	builtins.int*	octet
uint8	uint8_t	builtins.int*	octet
int16	int16_t	builtins.int*	short
uint16	uint16_t	builtins.int*	unsigned short
int32	int32_t	builtins.int*	long
uint32	uint32_t	builtins.int*	unsigned long
int64	int64_t	builtins.int*	long long

uint64	uint64_t	builtins.int*	unsigned long long
string	std::string	builtins.str	string
wstring	std::u16string	builtins.str	wstring

Every built-in-type can be used to define arrays:

Type name	C++	Python	DDS type
static array	std::array<T, N>	builtins.list*	T[N]
unbounded dynamic array	std::vector	builtins.list	sequence
bounded dynamic array	custom_class<T, N>	builtins.list*	sequence<T, N>
bounded string	std::string	builtins.str*	string

(*) All types that are more permissive than their ROS definition enforce the ROS constraints in range and length by software.

Example of message definition using arrays and bounded types:

```
int32[] unbounded_integer_array
int32[5] five_integers_array
int32[<=5] up_to_five_integers_array

string string_of_unbounded_size
string[<=10] up_to_ten_characters_string

string[<=5] up_to_five_unbounded_strings
string[<=10][] unbounded_array_of_strings_up_to_ten_characters_each
string[<=10][<=5] up_to_five_strings_up_to_ten_characters_each
```

Field names

Field names must be lowercase alphanumeric characters with underscores for separating words. They must start with an alphabetic character, and they must not end with an underscore or have two consecutive underscores.

Field default value

Default values can be set to any field in the message type. Currently default values are not supported for string arrays and complex types (i.e. types not present in the built-in-types table above; that applies to all nested messages).

Defining a default value is done by adding a third element to the field definition line, i.e:

```
fieldtype fieldname fielddefaultvalue
```

For example:

```
uint8 x 42
int16 y -2000
string full_name "John Doe"
int32[] samples [-200, -100, 0, 100, 200]
```

- string values must be defined in single ' or double " quotes

- currently string values are not escaped

Constants

Each constant definition is like a field description with a default value, except that this value can never be changed programmatically. This value assignment is indicated by use of an equal ‘=’ sign, e.g.

```
constanttype CONSTANTNAME=constantvalue
```

For example:

```
int32 X=123
int32 Y=-123
string FOO="foo"
string EXAMPLE='bar'
```

Topics

Send continuous data without expecting a reply.

Publish/Subscribe

A publish/subscribe system is one in which there are producers of data (publishers) and consumers of data (subscribers). The publishers and subscribers know how to contact each other through the concept of a “topic”, which is a common name so that the entities can find each other. For instance, when you create a publisher, you must also give it a string that is the name of the topic; the same goes for the subscriber. Any publishers and subscribers that are on the same topic name can directly communicate with each other. There may be zero or more publishers and zero or more subscribers on any particular topic. When data is published to the topic by any of the publishers, all subscribers in the system will receive the data. This system is also known as a “bus”, since it somewhat resembles a device bus from electrical engineering. This concept of a bus is part of what makes ROS 2 a powerful and flexible system. Publishers and subscribers can come and go as needed, meaning that debugging and introspection are natural extensions to the system. For instance, if you want to record data, you can use

the `ros2 bag record` command. Under the hood, `ros2 bag record` creates a new subscriber to whatever topic you tell it, without interrupting the flow of data to the other parts of the system.

Anonymous

Another fact mentioned in the introduction is that ROS 2 is “anonymous”. This means that when a subscriber gets a piece of data, it doesn’t generally know or care which publisher originally sent it (though it can find out if it wants). The benefit to this architecture is that publishers and subscribers can be swapped out at will without affecting the rest of the system.

Strongly-typed

Finally, the introduction also mentioned that the publish/subscribe system is “strongly-typed”. That has two meanings in this context:

The types of each field in a ROS message are typed, and that type is enforced at various levels. For instance, if the ROS message contains:

```
uint32 field1
string field2
```

1. Then the code will ensure that `field1` is always an unsigned integer and that `field2` is always a string.
2. The semantics of each field are well-defined. There is no automated mechanism to ensure this, but all of the core ROS types have strong semantics associated with them. For instance, the IMU message contains a 3-dimensional vector for the measured angular velocity, and each of the dimensions is specified to be in radians/second. Other interpretations should not be placed into the message.

Services

It is like a person waiting for a call and picks it up in a fraction of a second.

In ROS 2, a service refers to a remote procedure call. In other words, a node can make a remote procedure call to another node which will do a computation and return a result.

This structure is reflected in how a service message definition looks:

```
uint32 request
---
uint32 response
```

In ROS 2, services are expected to return quickly, as the client is generally waiting on the result. Services should never be used for longer running processes, in particular processes that might need to be preempted for exceptional situations. If you have a service that will be doing a long-running computation, consider using an [action](#) instead.

Services are identified by a service name, which looks much like a topic name (but is in a different namespace).

A service consists of two parts: the service server and the service client.

Service server

A service server is the entity that will accept a remote procedure request, and perform some computation on it. For instance, suppose the ROS 2 message contains the following:

```
uint32 a
uint32 b
---
uint32 sum
```

The service server would be the entity that receives this message, adds `a` and `b` together, and returns the `sum`.

Service client

A service client is an entity that will request a remote service server to perform a computation on its behalf. Following from the example above, the service client is the entity that creates the initial message containing `a` and `b`, and waits for the service server to compute the sum and return the result.

Unlike the service server, there can be arbitrary numbers of service clients using the same service name.

Actions

In ROS 2, an action refers to a long-running remote procedure call with feedback and the ability to cancel or preempt the goal. For instance, the high-level state machine running a robot may call an action to tell the navigation subsystem to travel to a waypoint, which may take several seconds (or minutes) to do. Along the way, the navigation subsystem can provide feedback on how far along it is, and the high-level state machine has the option to cancel or preempt the travel to that waypoint.

This structure is reflected in how an action message definition looks:

```
int32 request

---

int32 response

---

int32 feedback
```

In ROS 2, actions are expected to be long running procedures, as there is overhead in setting up and monitoring the connection. If you need a short running remote procedure call, consider using a [service](#) instead.

Actions are identified by an action name, which looks much like a topic name (but is in a different namespace).

An action consists of two parts: the action server and the action client.

Action server

The action server is the entity that will accept the remote procedure request and perform some procedure on it. It is also responsible for sending out feedback as the action progresses and should react to cancellation/preemption requests. For instance, consider an action to calculate the Fibonacci sequence with the following interface:

```
int32 order
```

```
---
```

```
int32[] sequence
```

```
---
```

```
int32[] sequence
```

The action server is the entity that receives this message, starts calculating the sequence up to `order` (providing feedback along the way), and finally returns a full result in `sequence`.

Action client

An action client is an entity that will request a remote action server to perform a procedure on its behalf. Following the example above, the action client is the entity that creates the initial message containing the `order`, and waits for the action server to compute the sequence and return it (with feedback along the way).

Unlike the action server, there can be arbitrary numbers of action clients using the same action name.

Parameters

Overview

Parameters in ROS 2 are associated with individual nodes. Parameters are used to configure nodes at startup (and during runtime), without changing the code. The lifetime of a parameter is tied to the lifetime of the node (though the node could implement some sort of persistence to reload values after restart).

Parameters are addressed by node name, node namespace, parameter name, and parameter namespace. Providing a parameter namespace is optional.

Each parameter consists of a key, a value, and a descriptor. The key is a string and the value is one of the following types: `bool`, `int64`, `float64`, `string`, `byte[]`, `bool[]`, `int64[]`, `float64[]` or `string[]`. By default all descriptors are empty, but can contain parameter descriptions, value ranges, type information, and additional constraints.

For a hands-on tutorial with ROS parameters see [Understanding parameters](#).

Parameters background

Declaring parameters

By default, a node needs to *declare* all of the parameters that it will accept during its lifetime. This is so that the type and name of the parameters are well-defined at node startup time, which reduces the chances of misconfiguration later on. See [Using parameters in a class \(C++\)](#) or [Using parameters in a class \(Python\)](#) for tutorials on declaring and using parameters from a node.

For some types of nodes, not all of the parameters will be known ahead of time. In these cases, the node can be instantiated with `allow_undeclared_parameters` set to `true`, which will allow parameters to be get and set on the node even if they haven't been declared.

Parameter types

Each parameter on a ROS 2 node has one of the pre-defined parameter types as mentioned in the Overview. By default, attempts to change the type of a declared parameter at runtime will fail. This prevents common mistakes, such as putting a boolean value into an integer parameter.

If a parameter needs to be multiple different types, and the code using the parameter can handle it, this default behavior can be changed. When the parameter is declared, it should be declared using a `ParameterDescriptor` with the `dynamic_typing` member variable set to `true`.

Parameter callbacks

A ROS 2 node can register two different types of callbacks to be informed when changes are happening to parameters. Both of the callbacks are optional.

The first is known as a “set parameter” callback, and can be set by calling `add_on_set_parameters_callback` from the node API. The callback is passed a list of immutable `Parameter` objects, and returns an `rcl_interfaces/msg/SetParametersResult`.

The main purpose of this callback is to give the user the ability to inspect the upcoming change to the parameter and explicitly reject the change.

The second type of callback is known as an “on parameter event” callback, and can be set by calling `on_parameter_event` from the parameter client APIs. The callback is passed an `rcl_interfaces/msg/ParameterEvent` object, and returns nothing. This callback will be called after all parameters in the input event have been declared, changed, or deleted.

The main purpose of this callback is to give the user the ability to react to changes from parameters that have successfully been accepted.

Interacting with parameters

ROS 2 nodes can perform parameter operations through node APIs as described in [Using parameters in a class \(C++\)](#) or [Using parameters in a class \(Python\)](#). External processes can perform parameter operations via parameter services that are created by default when a node is instantiated. The services that are created by default are:

- `/node_name/describe_parameters`: Uses a service type of `rcl_interfaces/srv/DescribeParameters`. Given a list of parameter names, returns a list of descriptors associated with the parameters.
- `/node_name/get_parameter_types`: Uses a service type of `rcl_interfaces/srv/GetParameterTypes`. Given a list of parameter names, returns a list of parameter types associated with the parameters.
- `/node_name/get_parameters`: Uses a service type of `rcl_interfaces/srv/GetParameters`. Given a list of parameter names, returns a list of parameter values associated with the parameters.
- `/node_name/list_parameters`: Uses a service type of `rcl_interfaces/srv/ListParameters`. Given an optional list of parameter prefixes, returns a list of the available parameters with that prefix. If the prefixes are empty, returns all parameters.
- `/node_name/set_parameters`: Uses a service type of `rcl_interfaces/srv/SetParameters`. Given a list of parameter names and values, attempts to set the parameters on the node. Returns a list of results from trying to set each parameter; some of them may have succeeded and some may have failed.
- `/node_name/set_parameters_atomically`: Uses a service type of `rcl_interfaces/srv/SetParametersAtomically`. Given a list of parameter names

and values, attempts to set the parameters on the node. Returns a single result from trying to set all parameters, so if one failed, all of them failed.

Setting initial parameter values when running a node

Initial parameter values can be set when running the node either through individual command-line arguments, or through YAML files. See [Setting parameters directly from the command line](#) for examples on how to set initial parameter values.

Setting initial parameter values when launching nodes

Initial parameter values can also be set when running the node through the ROS 2 launch facility. See [this document](#) for information on how to specify parameters via launch.

Manipulating parameter values at runtime

The `ros2 param` command is the general way to interact with parameters for nodes that are already running. `ros2 param` uses the parameter service API as described above to perform the various operations. See [this how-to guide](#) for details on how to use `ros2 param`.

Introspection with command line tools

ROS 2 includes a suite of command-line tools for introspecting a ROS 2 system.

Usage

The main entry point for the tools is the command `ros2`, which itself has various sub-commands for introspecting and working with nodes, topics, services, and more.

To see all available sub-commands run:

```
ros2 --help
```

Examples of sub-commands that are available include:

- `action`: Introspect/interact with ROS actions
- `bag`: Record/play a rosbag
- `component`: Manage component containers
- `daemon`: Introspect/configure the ROS 2 daemon
- `doctor`: Check ROS setup for potential issues
- `interface`: Show information about ROS interfaces
- `launch`: Run/introspect a launch file
- `lifecycle`: Introspect/manage nodes with managed lifecycles
- `multicast`: Multicast debugging commands
- `node`: Introspect ROS nodes
- `param`: Introspect/configure parameters on a node
- `pkg`: Introspect ROS packages
- `run`: Run ROS nodes
- `security`: Configure security settings
- `service`: Introspect/call ROS services
- `test`: Run a ROS launch test
- `topic`: Introspect/publish ROS topics
- `trace`: Tracing tools to get information on ROS nodes execution (only available on Linux)
- `wtf`: An alias for `doctor`

Example

To produce the typical talker-listener example using command-line tools, the `topic` sub-command can be used to publish and echo messages on a topic.

Publish messages in one terminal with:

```
ros2 topic pub /chatter std_msgs/msg/String "data: Hello world"
```

```
publisher: beginning loop
```

```
publishing #1: std_msgs.msg.String(data='Hello world')
```

```
publishing #2: std_msgs.msg.String(data='Hello world')
```

Echo messages received in another terminal with:

```
ros2 topic echo /chatter
```

```
data: Hello world
```

```
data: Hello world
```

Implementation

The source code for the `ros2` command is available at <https://github.com/ros2/ros2cli>.

The `ros2` tool has been implemented as a framework that can be extended via plugins. For example, the `sros2` package provides a `security` sub-command that is automatically detected by the `ros2` tool if the `sros2` package is installed.

Launch

A ROS 2 system typically consists of many nodes running across many different processes (and even different machines). While it is possible to manually start each of these nodes, it gets cumbersome quite quickly.

The launch system in ROS 2 is meant to automate the running of many nodes with a single command. It helps the user describe the configuration of their system and then executes it as described. The configuration of the system includes what programs to run, where to run them, what arguments to pass them, and ROS-specific conventions which make it easy to reuse components throughout the system by giving them each a different configuration. It is also responsible for monitoring the state of the processes launched, and reporting and/or reacting to changes in the state of those processes.

All of the above is specified in a “launch file”, which can be written in XML, YAML, or Python. This launch file can then be run using the `ros2 launch` command, and all of the nodes specified will be run.

Client libraries

Overview

Client libraries are the APIs that allow users to implement their ROS 2 code. Using client libraries, users gain access to ROS 2 concepts such as nodes, topics, services, etc. Client libraries come in a variety of programming languages so that users may write ROS 2 code in the language that is best-suited for their application. For example, you might prefer to write visualization tools in Python because it makes prototyping iterations faster, while for parts of your system that are concerned with efficiency, the nodes might be better implemented in C++.

Nodes written using different client libraries are able to share messages with each other because all client libraries implement code generators that provide users with the capability to interact with ROS 2 interface files in the respective language.

In addition to the language-specific communication tools, client libraries expose to users the core functionality that makes ROS “ROS”. For example, here is a list of functionality that can typically be accessed through a client library:

- Names and namespaces
- Time (real or simulated)
- Parameters
- Console logging
- Threading model
- Intra-process communication

Supported client libraries

The C++ client library (`rclcpp`) and the Python client library (`rclpy`) are both client libraries which utilize common functionality in `rcl`.

The `rclcpp` package

The ROS Client Library for C++ (`rclcpp`) is the user facing, C++ idiomatic interface which provides all of the ROS client functionality like creating nodes, publishers, and subscriptions. `rclcpp` builds on top of `rcl` and the `rosidl API`, and it is designed to be used with the C++ messages generated by `rosidl_generator_cpp`.

`rclcpp` makes use of all the features of C++ and C++17 to make the interface as easy to use as possible, but since it reuses the implementation in `rcl` it is able maintain a consistent behavior with the other client libraries that use the `rcl API`.

The `rclcpp` repository is located on GitHub at [ros2/rclcpp](https://github.com/ros2/rclcpp) and contains the `package rclcpp`. The generated `API` documentation is at <https://docs.ros.org/en/humble/p/rclcpp/>.

The `rclpy` package

The ROS Client Library for Python (`rclpy`) is the Python counterpart to the C++ client library. Like the C++ client library, `rclpy` also builds on top of the `rcl C API` for its implementation. The interface provides an idiomatic Python experience that uses native Python types and patterns like lists and context objects. By using the `rcl API` in the

implementation, it stays consistent with the other client libraries in terms of feature parity and behavior. In addition to providing Python idiomatic bindings around the `rcl` API and Python classes for each message, the Python client library takes care of the execution model, using `threading.Thread` or similar to run the functions in the `rcl` API.

Like C++ it generates custom Python code for each ROS message that the user interacts with, but unlike C++ it eventually converts the native Python message object into the C version of the message. All operations happen on the Python version of the messages until they need to be passed into the `rcl` layer, at which point they are converted into the plain C version of the message so it can be passed into the `rcl` C API. This is avoided if possible when communicating between publishers and subscriptions in the same process to cut down on the conversion into and out of Python.

The `rclpy` repository is located on GitHub at [ros2/rclpy](https://github.com/ros2/rclpy) and contains the package `rclpy`. The generated API documentation is at <https://docs.ros.org/en/humble/p/rclpy/>.

Community-maintained

While the C++ and Python client libraries are maintained by the core ROS 2 team, members of the ROS 2 community maintain additional client libraries:

- [Ada](#) This is a set of packages (binding to `rcl`, message generator, binding to `tf2`, examples and tutorials) that allows the writing of Ada applications for ROS 2.
- [C rclc](#) does not put a layer on top of `rcl` but complements `rcl` to make `rcl+rclc` a feature-complete client library in C. See micro.ros.org for tutorials.
- [JVM and Android](#) Java and Android bindings for ROS 2.
- [.NET Core, UWP and C#](#) This is a collection of projects (bindings, code generator, examples and more) for writing ROS 2 applications for .NET Core and .NET Standard.
- [Node.js](#) `rclnodejs` is a Node.js client for ROS 2. It provides a simple and easy JavaScript API for ROS 2 programming.

- [Rust](#) This is a set of projects (the rclrs client library, code generator, examples and more) that enables developers to write ROS 2 applications in Rust.
- [Flutter and Dart](#) Flutter and Dart bindings for ROS 2.

Older, unmaintained client libraries are:

- [C#](#)
- [Objective C and iOS](#)
- [Zig](#)

Common functionality: [rcl](#)

Most of the functionality found in a client library is not specific to the programming language of the client library. For example, the behavior of parameters and the logic of namespaces should ideally be the same across all programming languages. Because of this, rather than implementing the common functionality from scratch, client libraries make use of a common core ROS Client Library (RCL) interface that implements logic and behavior of ROS concepts that is not language-specific. As a result, client libraries only need to wrap the common functionality in the RCL with foreign function interfaces. This keeps client libraries thinner and easier to develop. For this reason the common RCL functionality is exposed with C interfaces as the C language is typically the easiest language for client libraries to wrap.

In addition to making the client libraries light-weight, an advantage of having the common core is that the behavior between languages is more consistent. If any changes are made to the logic/behavior of the functionality in the core RCL – namespaces, for example – all client libraries that use the RCL will have these changes reflected. Furthermore, having the common core means that maintaining multiple client libraries becomes less work when it comes to bug fixes.

The API documentation for [rcl](#) can be found [here](#).

Language-specific functionality

Client library concepts that require language-specific features/properties are not implemented in the RCL but instead are implemented in each client library. For example, threading models used by “spin” functions will have implementations that are specific to the language of the client library.

Demo

For a walkthrough of the message exchange between a publisher using `rclpy` and a subscription using `rclcpp`, we encourage you to watch [this ROSCon talk](#) starting at 17:25 ([see the slides here](#))